

Helm Templating

Loops, Conditions & Practical Patterns

What you'll learn

- Go template basics used by Helm
- Variables, scoping and the `.` (dot)
- `if/else`, logical ops, `required` & `fail`
- `range` loops over lists/maps/files
- Includes/partials (`_helpers.tpl`) and `dict`
- Indentation & YAML-safe rendering (`toYaml`, `nindent`)
- Powerful Sprig functions you'll use daily
- Advanced patterns: `tpl`, conditionally emitting resources, `map` → `env`, `list` → `ports`

Helm's Template Engine

- **Go templates** + **Sprig** functions + Helm helpers
- Templates live in `templates/` and render to Kubernetes YAML
- Values flow from: chart `values.yaml` → user `-f` files → `--set`

Chart + Values --(Go templates)--> Rendered YAML --(kubectl)--> Cluster

The Dot (.) & Variables

- `.` = current context (object you're templating with)
- Root context often saved as `$` so you don't lose it inside loops

```
{{- $ := . -}}           {{/* root */}}  
{{- $name := include "app.fullname" . -}}  
metadata:  
  name: {{ $name }}
```

Assign variables

```
{{- $port := default 80 .Values.service.port -}}  
{{- $image := printf "%s:%s" .Values.image.repository (.Values.image.tag | default "latest") -}}
```

Conditions

```
{{- if .Values.ingress.enabled }}  
# render an Ingress here  
{{- end }}  
  
{{- if and .Values.tls.enabled (ne .Values.service.type "LoadBalancer") }}  
# secure internal-only config  
{{- else }}  
# fallback  
{{- end }}
```

Common funcs: `and` , `or` , `not` , `eq` , `ne` , `lt` , `gt` , `default` .

Fail fast

```
{{- required "image.repository is required" .Values.image.repository -}}  
{{- if not .Values.securityContext.runAsNonRoot -}}  
{{- fail "runAsNonRoot must be true in production" -}}  
{{- end -}}
```

Ranging (Loops)

Iterate arrays/lists:

```
{{- range .Values.ports }}  
- containerPort: {{ . }}  
{{- end }}
```

Iterate maps (key/value):

```
{{- range $k, $v := .Values.annotations }}  
  {{ $k }}: {{ $v | quote }}  
{{- end }}
```

With index:

```
{{- range $i, $c := .Values.extraContainers }}  
- name: sidecar-{{ $i }}  
  image: {{ $c.image }}  
{{- end }}
```

YAML-Safe Indentation

`toYaml` → converts maps/lists to YAML

`indent/nindent` → indent by N spaces (`nindent` trims leading newline)

```
labels:
  {{- include "app.labels" . | nindent 2 }}

env:
  {{- toYaml .Values.env | nindent 2 }}
```

Avoid manual spacing. Pipe to `nindent` for proper nesting.

Includes & Partials (`_helpers.tpl`)

Keep templates DRY.

`templates/_helpers.tpl`

```
{{- define "app.name" -}}{{ .Chart.Name }}{{- end -}}
{{- define "app.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end -}}
{{- define "app.labels" -}}
app.kubernetes.io/name: {{ include "app.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
{{- end -}}
```

Use in other templates:

```
metadata:  
  name: {{ include "app.fullname" . }}  
  labels:  
    {{- include "app.labels" . | nindent 4 }}
```

Passing Multiple Values into `include`

`include` only accepts one argument → pass a map with `dict`.

```
{{- $ctx := dict "root" . "name" "metrics" "port" 9090 -}}  
{{ include "app.container" $ctx }}
```

`_helpers.tpl`

```
{{- define "app.container" -}}  
{{- $ := .root -}}  
- name: {{ .name }}  
  image: {{ $.Values.metrics.image }}  
  ports:  
    - containerPort: {{ .port }}  
{{- end -}}
```

From Values → Env Vars (map → list)

values.yaml

```
env:  
  LOG_LEVEL: info  
  FEATURE_X: "true"
```

deployment snippet

```
env:  
  {{- range $k, $v := .Values.env }}  
    - name: {{ $k }}  
      value: {{ $v | quote }}  
  {{- end }}
```

Rendering Lists (ports, volumeMounts, tolerations)

```
ports:
{{- range .Values.container.ports }}
  - name: {{ .name }}
    containerPort: {{ .containerPort }}
{{- end }}

volumeMounts:
{{- toYaml .Values.volumeMounts | nindent 2 }}

tolerations:
{{- toYaml .Values.tolerations | nindent 2 }}
```

Handling Optional Sections

```
{{- with .Values.resources }}  
resources:  
  {{- toYaml . | nindent 2 }}  
{{- end }}  
  
{{- with .Values.nodeSelector }}  
nodeSelector:  
  {{- toYaml . | nindent 2 }}  
{{- end }}
```

`with` changes the dot (`.`) to the inner object when non-empty.

String Tricks & Pipelines

- Build strings: `printf`, `cat`, `join`, `replace`, `trimSuffix`, `trunc`
- Quote carefully: `quote`, `squote`
- Hashes for annotations: `sha256sum`

```
annotations:  
  checksum/config: {{ .Values.config | toYaml | sha256sum }}  
  
image: {{ printf "%s:%s" .Values.image.repository (.Values.image.tag | default "1.0.0") }}
```

Use pipelines to compose readable transformations.

Converting Data

- `toYaml`, `toJson`
- `fromYaml`, `fromJson` (parse strings into structures)
- `b64enc`, `b64dec`

```
{{- $cfg := fromYaml (.Files.Get "config/app.yaml") -}}  
{{- range $k, $v := $cfg.extraEnv }}  
- name: {{ $k }}  
  value: {{ $v | quote }}  
{{- end }}
```

Using `.Files` & Templating Files

- Bundle files under chart root (not in `templates/`)
- Access with `.Files.Get` , `.Files.Glob` , `.Files.GetBytes`

```
data:  
  app.yaml: |-  
    {{ (.Files.Get "config/app.yaml") | nindent 4 }}
```

Templating strings from values with `tpl`:

```
{{- $raw := .Values.sidecar.tplCommand -}}  
command: {{ tpl $raw . | toJson }}
```

values.yaml

```
sidecar:  
  tplCommand: |
```

Logic Helpers (Sprig)

- List & set ops: `has`, `mustHas`, `uniq`, `compact`, `pluck`
- Math/time: `add`, `until`, `untilStep`, `now`, `date`
- Conditionals: `ternary`, `coalesce`, `default`

```
{{- $replicas := ternary 3 1 .Values.ha.enabled -}}
```

Generate N ports

```
{{- range $i := untilStep 8000 8005 1 }}  
- containerPort: {{ $i }}  
{{- end }}
```

Guarding for Cluster Versions

Prefer feature flags, but you can branch on capabilities:

```
apiVersion: {{- if .Capabilities.APIVersions.Has "policy/v1/PodDisruptionBudget" -}}policy/v1{{- else -}}policy/v1beta1{{- end -}}
```

Validate with:

```
helm template myapp ./chart --kube-version 1.26 > out.yaml  
helm template myapp ./chart --kube-version 1.30 > out.yaml
```

Conditionally Emitting Whole Resources

```
{{- if .Values.cron.enabled -}}  
apiVersion: batch/v1  
kind: CronJob  
metadata:  
  name: {{ include "app.fullname" . }}  
... # spec  
{{- end -}}
```

This pattern keeps the chart tidy and avoids half-rendered objects.

Common Gotchas (and Fixes)

- **Indentation errors** → always pipe to `nindent`
- **Losing root context** → save `$ := .` before deep loops
- **Booleans vs strings** → use `--set-string` when exact value needed
- **63-char names** → `trunc 63 | trimSuffix "-"`
- **Selectors mismatch** → centralize labels in helpers and reuse
- **Empty lists rendering** → guard with `with` or `if` blocks

Example: Full Deployment Snippet

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "app.fullname" . }}
  labels:
    {{- include "app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount | default 1 }}
  selector:
    matchLabels:
      {{- include "app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "app.selectorLabels" . | nindent 8 }}
    annotations:
      {{- range $k, $v := .Values.podAnnotations }}
      {{ $k }}: {{ $v | quote }}
      {{- end }}
  spec:
```

```
containers:
  - name: app
    image: {{ printf "%s:%s" .Values.image.repository (.Values.image.tag | default .Chart.AppVersion) }}
    ports:
      {{- range .Values.ports }}
        - containerPort: {{ . }}
      {{- end }}
    env:
      {{- range $k, $v := .Values.env }}
        - name: {{ $k }}
          value: {{ $v | quote }}
      {{- end }}
    resources:
      {{- toYaml .Values.resources | nindent 12 }}
  {{- with .Values.nodeSelector }}
nodeSelector:
  {{- toYaml . | nindent 8 }}
  {{- end }}
```

Patterns You'll Reuse

- Map → annotations/labels/env
- List → ports/volumes/tolerations
- `with` for optional blocks
- `tpl` for templated strings from values
- `include` + `dict` for passing multiple args
- **Checksum annotations** to trigger rollouts when config changes

Hands-On Mini-Lab (15 min)

1. Add `_helpers.tpl` with `name`, `fullname`, `labels`
2. Convert hard-coded labels to `include "app.labels"` .
3. Loop `values.env` into container env vars
4. Add flag `ingress.enabled` and conditionally render Ingress
5. Use `tpl` to template a sidecar command from values

Checklist

-  Use helpers to keep templates DRY
-  Guard optional sections with `with` / `if`
-  Always `toYaml | nindent` for blocks
-  Keep root context in `$` when ranging
-  Validate renders on multiple kube versions

Appendix: Quick Reference

- **Conditionals:** `if`, `else`, `and`, `or`, `not`, `eq`, `ne`, `default`, `ternary`
- **Loops:** `range`, `until`, `untilStep`
- **Data:** `toYaml`, `fromYaml`, `toJson`, `fromJson`, `b64enc`
- **Strings:** `printf`, `replace`, `trim`, `trunc`, `quote`
- **Flow:** `include`, `tpl`, `required`, `fail`, `dict`, `list`
- **Files:** `.Files.Get`, `.Files.Glob`, `.Files.GetBytes`